

ANTI-PATTERNS EM FACTORY E DI: GUIA PRÁTICO DE DESCOBERTA

Aprendendo a Reconhecer e Corrigir Problemas de Design

Sobre Este Material

Este documento complementar foi criado especificamente para desenvolver em você a habilidade crítica de reconhecer problemas de design em código que aparentemente funciona. Esta é uma competência fundamental que distingue desenvolvedores júnior de desenvolvedores sênior: a capacidade de olhar para código funcional e identificar problemas estruturais que causarão dificuldades futuras.

Você está prestes a explorar cinco cenários reais de código problemático. Cada cenário representa um anti-pattern comum que desenvolvedores encontram (e frequentemente criam sem perceber) quando estão aprendendo sobre factories e dependency injection. O objetivo não é simplesmente memorizar listas de problemas, mas sim desenvolver sua intuição para identificar code smells e entender profundamente por que certas abordagens, embora funcionem inicialmente, criam dívida técnica significativa.

Como Usar Este Material

Este material foi desenhado para ser trabalhado de forma ativa, não passiva. Para cada cenário apresentado, você deve seguir um processo deliberado de descoberta. Primeiro, leia o código completo tentando entender o que ele faz funcionalmente. Execute o código se possível para ver que ele realmente funciona. Depois, antes de ler a análise fornecida, tente responder às perguntas investigativas por conta própria. Anote suas respostas e raciocínio. Só então compare suas conclusões com a análise detalhada que fornecemos.

Se você está estudando em grupo, discuta cada cenário antes de ler a solução. Debates entre colegas frequentemente revelam perspectivas diferentes e levam a insights mais profundos. Se você está estudando sozinho, tente explicar os problemas em voz alta como se estivesse ensinando outra pessoa. Esta técnica de verbalização força você a organizar seus pensamentos de forma clara.

Lembre-se que o objetivo não é acertar todas as respostas imediatamente. O objetivo é desenvolver o processo mental de análise crítica de código. Errar faz parte fundamental do aprendizado, então não se frustre se você não identificar todos os problemas na primeira leitura. Com prática, sua habilidade de reconhecimento de anti-patterns se aprimorará significativamente.

CENÁRIO 1: O DEUS DO OLIMPO (God Factory Anti-Pattern)

O Código Problemático

Imagine que você está revisando o código de um colega que está desenvolvendo um sistema de gerenciamento de recursos para uma aplicação empresarial. Ele criou uma factory "universal" que centraliza toda a criação de objetos do sistema. Veja o código dele:


```
using Microsoft.Extensions.DependencyInjection;
using System;

// Factory "universal" que cria tudo
public class ApplicationFactory
{
    private readonly IServiceProvider _serviceProvider;
    private readonly IConfiguration _configuration;
    private readonly ILogger _logger;

    public ApplicationFactory(
        IServiceProvider serviceProvider,
        IConfiguration configuration,
        ILogger logger)
    {
        _serviceProvider = serviceProvider;
        _configuration = configuration;
        _logger = logger;
    }

    // Método gigante que cria qualquer tipo de objeto do sistema
    public object Create(string type, Dictionary<string, object> parameters = null)
    {
        _logger.Log($"Criando objeto do tipo: {type}");

        return type.ToLower() switch
        {
            // Criação de notificadores
            "emailnotifier" => CreateEmailNotifier(parameters),
            "smsnotifier" => CreateSmsNotifier(parameters),
            "pushnotifier" => CreatePushNotifier(parameters),

            // Criação de processadores de pagamento
            "creditcardprocessor" => CreateCreditCardProcessor(parameters),
            "boletoprocessor" => CreateBoletoProcessor(parameters),
            "pixprocessor" => CreatePixProcessor(parameters),

            // Criação de geradores de relatório
            "pdfreportgenerator" => CreatePdfReportGenerator(parameters),
            "excelreportgenerator" => CreateExcelReportGenerator(parameters),
            "htmlreportgenerator" => CreateHtmlReportGenerator(parameters),

            // Criação de repositórios
            "userrepository" => CreateUserRepository(parameters),
            "orderrepository" => CreateOrderRepository(parameters),
            "productrepository" => CreateProductRepository(parameters),
```

```

// Criação de validadores
"emailvalidator" => CreateEmailValidator(parameters),
"cpfvalidator" => CreateCpfValidator(parameters),
"creditcardvalidator" => CreateCreditCardValidator(parameters),

// Criação de serviços de integração
"paymentgatewayservice" => CreatePaymentGatewayService(parameters),
"shippingservice" => CreateShippingService(parameters),
"inventoryservice" => CreateInventoryService(parameters),

// ... e assim por diante para TODOS os tipos do sistema

_ => throw new ArgumentException($"Tipo desconhecido: {type}")
};
}

```

```
private object CreateEmailNotifier(Dictionary<string, object> parameters)
```

```

{
    var smtpServer = _configuration["Email:SmtpServer"];
    var port = int.Parse(_configuration["Email:Port"]);
    var username = _configuration["Email:Username"];
    var password = _configuration["Email:Password"];

```

```

// Lógica complexa de criação

```

```

var notifier = new EmailNotifier(smtpServer, port);
notifier.SetCredentials(username, password);

```

```

if (parameters?.ContainsKey("useSsl") == true)
{
    notifier.EnableSsl((bool)parameters["useSsl"]);
}

```

```

return notifier;

```

```

}

```

```
private object CreateCreditCardProcessor(Dictionary<string, object> parameters)
```

```

{
    var gateway = _configuration["Payment:Gateway"];
    var apiKey = _configuration["Payment:ApiKey"];

    var processor = new CreditCardProcessor(gateway, apiKey);

```

```

if (parameters?.ContainsKey("timeout") == true)
{
    processor.SetTimeout((int)parameters["timeout"]);
}

```

```

        return processor;
    }

    private object CreateUserRepository(Dictionary<string, object> parameters)
    {
        var connectionString = _configuration.GetConnectionString("Default");
        var repository = new UserRepository(connectionString);

        if (parameters?.ContainsKey("cacheEnabled") == true)
        {
            repository.EnableCache((bool)parameters["cacheEnabled"]);
        }

        return repository;
    }

    // ... dezenas de outros métodos Create[TipoEspecífico] ...
    // (imagine mais 15-20 métodos similares aqui)
}

// Uso no código cliente
public class OrderService
{
    private readonly ApplicationFactory _factory;

    public OrderService(ApplicationFactory factory)
    {
        _factory = factory;
    }

    public void ProcessOrder(Order order)
    {
        // Cliente precisa saber strings mágicas
        var paymentProcessor = _factory.Create("creditcardprocessor") as CreditCardProcessor;
        var emailNotifier = _factory.Create("emailnotifier") as EmailNotifier;
        var orderRepository = _factory.Create("orderrepository") as OrderRepository;

        // Processar pedido...
    }
}

// Classes de exemplo (simplificadas)
public interface IConfiguration
{
    string this[string key] { get; }
    string GetConnectionString(string name);
}

```

```

    }

    public interface ILogger
    {
        void Log(string message);
    }

    public class EmailNotifier
    {
        public EmailNotifier(string smtpServer, int port) { }
        public void SetCredentials(string username, string password) { }
        public void EnableSsl(bool enabled) { }
    }

    public class CreditCardProcessor
    {
        public CreditCardProcessor(string gateway, string apiKey) { }
        public void SetTimeout(int seconds) { }
    }

    public class UserRepository
    {
        public UserRepository(string connectionString) { }
        public void EnableCache(bool enabled) { }
    }

    public class Order { }

```

Perguntas Investigativas

Antes de continuar lendo, pare e tente responder honestamente estas perguntas. Anote suas respostas para comparar depois com a análise detalhada.

Primeiro, conte quantas responsabilidades diferentes esta classe `ApplicationFactory` tem. Liste cada responsabilidade que você identificar. Pense não apenas em termos de funcionalidades, mas em termos de razões pelas quais esta classe poderia precisar ser modificada no futuro.

Segundo, imagine que você precisa adicionar um novo tipo de objeto ao sistema, digamos um novo tipo de validador chamado `PhoneValidator`. Quantos lugares no código você precisaria modificar? Liste especificamente cada arquivo e cada método que requereria mudanças.

Terceiro, considere testabilidade. Se você quisesse escrever um teste unitário para `OrderService`, como você mockaria a `ApplicationFactory`? Quais dificuldades você encontraria ao tentar fazer isso? Pense sobre o que você precisaria fazer para garantir que o mock retorne os tipos corretos.

Quarto, pense sobre manutenibilidade. Se o método de criação de `EmailNotifier` mudasse para exigir um parâmetro adicional, o que aconteceria? Quantos desenvolvedores diferentes poderiam estar modificando esta

classe simultaneamente e quais conflitos de merge você anteciparia?

Quinto, analise o acoplamento. Quantas classes concretas diferentes a ApplicationFactory conhece diretamente? Como isso afeta a capacidade de evoluir diferentes partes do sistema independentemente?

Sintomas Observáveis

Quando você se depara com um God Factory no código real, certos sintomas se tornam aparentes muito rapidamente. O primeiro e mais óbvio sintoma é o tamanho do arquivo. Um God Factory tipicamente tem centenas ou até milhares de linhas de código em um único arquivo. Quando você abre este arquivo em seu editor, precisa rolar por muito tempo para ver todo o conteúdo.

O segundo sintoma é a frequência de conflitos de merge em sistemas de controle de versão. Se você está trabalhando em uma equipe e usa Git, você notará que este arquivo específico está constantemente envolvido em conflitos de merge. Isso acontece porque múltiplos desenvolvedores precisam modificar o mesmo arquivo para adicionar seus respectivos tipos de objetos.

O terceiro sintoma é a dificuldade em entender o que a classe faz. Quando alguém novo na equipe pergunta "o que esta classe faz?", a resposta é sempre vaga e abrangente: "ela cria coisas" ou "ela é uma factory universal". Não há uma descrição concisa e específica da responsabilidade.

O quarto sintoma aparece nos testes. Os testes que dependem desta factory são complexos e frágeis. Eles precisam mockar uma factory gigante que pode retornar dezenas de tipos diferentes, e frequentemente quebram quando mudanças são feitas em partes completamente não relacionadas do código.

O quinto sintoma é a resistência a mudanças. Desenvolvedores começam a hesitar em modificar esta classe porque ela é tão grande e complexa que há medo de quebrar algo inesperadamente. Esta hesitação leva a workarounds e código duplicado em outros lugares, piorando ainda mais a qualidade geral do código.

Por Que É Problemático

O God Factory viola fundamentalmente o princípio de responsabilidade única, que é o primeiro e talvez mais importante princípio SOLID. Esta classe tem literalmente dezenas de responsabilidades diferentes. Ela é responsável por criar notificadores, processadores de pagamento, geradores de relatório, repositórios, validadores e serviços de integração. Cada uma dessas categorias por si só justificaria uma factory separada.

Quando uma classe tem muitas responsabilidades, ela tem muitas razões para mudar. Toda vez que qualquer parte do sistema precisa de um novo tipo de objeto, esta classe precisa ser modificada. Isso cria um ponto central de falha no sistema. Uma mudança mal feita em qualquer um dos métodos de criação pode potencialmente afetar qualquer parte do sistema que use a factory.

O God Factory também viola severamente o princípio Open/Closed. A classe não está aberta para extensão mas fechada para modificação. Pelo contrário, ela está completamente fechada para extensão (você não pode adicionar novos tipos sem modificá-la) e completamente aberta para modificação (você é forçado a modificá-la constantemente). Isso é exatamente o oposto do que queremos.

Do ponto de vista de testabilidade, o God Factory é um pesadelo. Quando você quer testar uma classe que depende da factory, você precisa mockar uma interface gigante que pode retornar qualquer tipo de objeto. Você

não pode simplesmente mockar os métodos específicos que sua classe sob teste usa porque tudo passa pelo mesmo método `Create` genérico. Isso leva a mocks complicados e testes frágeis que quebram facilmente.

O God Factory também cria acoplamento massivo. Esta única classe conhece intimamente dezenas ou centenas de classes concretas do sistema. Qualquer mudança em qualquer uma dessas classes concretas potencialmente requer mudanças na factory. Isso cria uma teia densa de dependências que torna o sistema rígido e difícil de modificar.

Finalmente, o God Factory dificulta muito o trabalho em equipe. Se você tem cinco desenvolvedores trabalhando em cinco features diferentes, mas todos precisam adicionar algo à mesma factory gigante, você tem um gargalo de integração contínua. Conflitos de merge se tornam frequentes e resolução desses conflitos consome tempo valioso de desenvolvimento.

A Refatoração: Passo a Passo

Vamos transformar este God Factory em uma arquitetura bem estruturada com múltiplas factories especializadas, cada uma com responsabilidade única e clara. Este processo de refatoração acontecerá em várias etapas deliberadas, e em cada etapa explicarei não apenas o que estamos fazendo, mas principalmente por que estamos fazendo dessa forma.

Passo 1: Identificar Domínios de Responsabilidade

O primeiro passo em qualquer refatoração de God Factory é identificar os diferentes domínios de responsabilidade. Olhando para nosso código problemático, podemos claramente identificar seis domínios distintos: notificações, processamento de pagamento, geração de relatórios, acesso a dados (repositórios), validação e serviços de integração externa.

Cada um desses domínios representa um conjunto coeso de funcionalidades que naturalmente pertencem juntas. Notificadores de email, SMS e push são todos parte do mesmo domínio de notificações. Processadores de cartão de crédito, boleto e PIX são todos parte do domínio de pagamentos. Esta coesão é nosso guia para dividir a responsabilidade.

Passo 2: Criar Interfaces de Factory Especializadas

Agora vamos criar uma interface separada para cada domínio. Cada interface será focada e específica, definindo claramente quais tipos de objetos ela é responsável por criar.

csharp

// Factory especializada para notificações

public interface INotificationFactory

```
{  
    INotifier CreateEmailNotifier();  
    INotifier CreateSmsNotifier();  
    INotifier CreatePushNotifier();  
}
```

// Factory especializada para pagamentos

public interface IPaymentProcessorFactory

```
{  
    IPaymentProcessor CreateCreditCardProcessor();  
    IPaymentProcessor CreateBoletoProcessor();  
    IPaymentProcessor CreatePixProcessor();  
}
```

// Factory especializada para relatórios

public interface IReportGeneratorFactory

```
{  
    IReportGenerator CreatePdfGenerator();  
    IReportGenerator CreateExcelGenerator();  
    IReportGenerator CreateHtmlGenerator();  
}
```

// Factory especializada para repositórios

public interface IRepositoryFactory

```
{  
    IUserRepository CreateUserRepository();  
    IOrderRepository CreateOrderRepository();  
    IProductRepository CreateProductRepository();  
}
```

// Factory especializada para validadores

public interface IValidatorFactory

```
{  
    IValidator CreateEmailValidator();  
    IValidator CreateCpfValidator();  
    IValidator CreateCreditCardValidator();  
}
```

// Factory especializada para serviços de integração

public interface IIntegrationServiceFactory

```
{  
    IPaymentGatewayService CreatePaymentGatewayService();  
    IShippingService CreateShippingService();  
}
```

```
IInventoryService CreateInventoryService();  
}
```

Observe que cada interface tem um nome descritivo que comunica claramente sua responsabilidade.

INotificationFactory é obviamente responsável por criar notificadores. Não há ambiguidade. Além disso, cada método retorna uma interface apropriada (INotifier, IPaymentProcessor, etc.) ao invés de object. Isso fornece type safety em tempo de compilação.

Passo 3: Implementar Factories Concretas

Agora implementamos cada factory especializada. Vou mostrar a implementação completa de uma delas e você verá o padrão que se aplica a todas as outras.

```
csharp
```

```
public class NotificationFactory : INotificationFactory
{
    private readonly IServiceProvider _serviceProvider;
    private readonly IConfiguration _configuration;
    private readonly ILogger<NotificationFactory> _logger;

    // Constructor Injection de dependências específicas desta factory
    public NotificationFactory(
        IServiceProvider serviceProvider,
        IConfiguration configuration,
        ILogger<NotificationFactory> logger)
    {
        _serviceProvider = serviceProvider;
        _configuration = configuration;
        _logger = logger;
    }

    public INotifier CreateEmailNotifier()
    {
        _logger.LogInformation("Criando EmailNotifier");

        // Resolver do container DI ou criar com configuração
        var notifier = _serviceProvider.GetRequiredService<EmailNotifier>();

        // Configurações específicas se necessário
        var smtpServer = _configuration["Email:SmtpServer"];
        if (!string.IsNullOrEmpty(smtpServer))
        {
            notifier.ConfigureSmtpServer(smtpServer);
        }

        return notifier;
    }

    public INotifier CreateSmsNotifier()
    {
        _logger.LogInformation("Criando SmsNotifier");
        return _serviceProvider.GetRequiredService<SmsNotifier>();
    }

    public INotifier CreatePushNotifier()
    {
        _logger.LogInformation("Criando PushNotifier");
        return _serviceProvider.GetRequiredService<PushNotifier>();
    }
}
```

```
}  
}
```

Veja como esta factory é focada. Ela conhece apenas três classes concretas, todas do mesmo domínio. Se precisarmos adicionar um novo tipo de notificador, modificamos apenas esta factory, não afetando nenhum outro domínio. Se o desenvolvedor trabalhando em pagamentos precisa adicionar um novo processador, ele modifica PaymentProcessorFactory, não havendo conflito com nosso trabalho em notificações.

Passo 4: Registrar Factories no Container DI

Agora precisamos registrar todas essas factories no container de dependency injection. Este registro normalmente aconteceria na configuração inicial da aplicação, tipicamente no Program.cs ou Startup.cs.

```
csharp  
  
// Em Program.cs ou Startup.cs  
public void ConfigureServices(IServiceCollection services)  
{  
    // Registrar implementações concretas  
    services.AddTransient<EmailNotifier>();  
    services.AddTransient<SmsNotifier>();  
    services.AddTransient<PushNotifier>();  
  
    services.AddTransient<CreditCardProcessor>();  
    services.AddTransient<BoletoProcessor>();  
    services.AddTransient<PixProcessor>();  
  
    // ... registrar outras implementações concretas  
  
    // Registrar factories especializadas  
    services.AddSingleton<INotificationFactory, NotificationFactory>();  
    services.AddSingleton<IPaymentProcessorFactory, PaymentProcessorFactory>();  
    services.AddSingleton<IReportGeneratorFactory, ReportGeneratorFactory>();  
    services.AddSingleton<IRepositoryFactory, RepositoryFactory>();  
    services.AddSingleton<IValidatorFactory, ValidatorFactory>();  
    services.AddSingleton<IIntegrationServiceFactory, IntegrationServiceFactory>();  
}
```

Note que as factories são registradas como Singleton porque elas não mantêm estado e apenas coordenam a criação de outros objetos. Os produtos criados pelas factories (EmailNotifier, CreditCardProcessor, etc.) são registrados como Transient porque cada uso tipicamente quer uma instância nova.

Passo 5: Refatorar Código Cliente

Finalmente, refatoramos o código que usava o God Factory para usar as factories especializadas apropriadas.

```
csharp
```

// ANTES: Cliente acoplado ao God Factory

```
public class OrderService
{
    private readonly ApplicationFactory _factory;

    public OrderService(ApplicationFactory factory)
    {
        _factory = factory;
    }

    public void ProcessOrder(Order order)
    {
        // Strings mágicas e casts feios
        var paymentProcessor = _factory.Create("creditcardprocessor") as CreditCardProcessor;
        var emailNotifier = _factory.Create("emailnotifier") as EmailNotifier;
        var orderRepository = _factory.Create("orderrepository") as OrderRepository;

        // Processar pedido...
    }
}
```

// DEPOIS: Cliente usa apenas factories relevantes

```
public class OrderService
{
    private readonly IPaymentProcessorFactory _paymentFactory;
    private readonly INotificationFactory _notificationFactory;
    private readonly IRepositoryFactory _repositoryFactory;

    // Constructor injection deixa dependências explícitas
    public OrderService(
        IPaymentProcessorFactory paymentFactory,
        INotificationFactory notificationFactory,
        IRepositoryFactory repositoryFactory)
    {
        _paymentFactory = paymentFactory ?? throw new ArgumentNullException(nameof(paymentFactory));
        _notificationFactory = notificationFactory ?? throw new ArgumentNullException(nameof(notificationFactory));
        _repositoryFactory = repositoryFactory ?? throw new ArgumentNullException(nameof(repositoryFactory));
    }

    public void ProcessOrder(Order order)
    {
        // Type-safe, sem strings mágicas, sem casts
        var paymentProcessor = _paymentFactory.CreateCreditCardProcessor();
        var emailNotifier = _notificationFactory.CreateEmailNotifier();
        var orderRepository = _repositoryFactory.CreateOrderRepository();
    }
}
```

```
// Processar pedido com objetos fortemente tipados
```

```
}  
}
```

Veja a melhoria dramática na qualidade do código cliente. As dependências agora são explícitas no construtor. Qualquer pessoa lendo este código sabe imediatamente que `OrderService` precisa de três factories específicas. Não há strings mágicas que podem ser digitadas incorretamente. Não há casts perigosos que podem falhar em runtime. O compilador garante type safety completo.

Passo 6: Melhorar Testabilidade

Agora vamos ver como a testabilidade melhorou dramaticamente com esta refatoração.

```
csharp
```

```
using Xunit;
using Moq;

public class OrderServiceTests
{
    [Fact]
    public void ProcessOrder_WithValidOrder_CallsAppropriateServices()
    {
        // Arrange: Mockar apenas as factories que OrderService realmente usa
        var mockPaymentFactory = new Mock<IPaymentProcessorFactory>();
        var mockNotificationFactory = new Mock<INotificationFactory>();
        var mockRepositoryFactory = new Mock<IRepositoryFactory>();

        // Criar mocks dos produtos que serão retornados
        var mockPaymentProcessor = new Mock<IPaymentProcessor>();
        var mockNotifier = new Mock<INotifier>();
        var mockRepository = new Mock<IOrderRepository>();

        // Configurar factories para retornar mocks
        mockPaymentFactory
            .Setup(f => f.CreateCreditCardProcessor())
            .Returns(mockPaymentProcessor.Object);

        mockNotificationFactory
            .Setup(f => f.CreateEmailNotifier())
            .Returns(mockNotifier.Object);

        mockRepositoryFactory
            .Setup(f => f.CreateOrderRepository())
            .Returns(mockRepository.Object);

        // Criar OrderService com mocks injetados
        var service = new OrderService(
            mockPaymentFactory.Object,
            mockNotificationFactory.Object,
            mockRepositoryFactory.Object
        );

        var order = new Order { /* dados do pedido */ };

        // Act
        service.ProcessOrder(order);

        // Assert: Verificar que métodos corretos foram chamados
        mockPaymentFactory.Verify(f => f.CreateCreditCardProcessor(), Times.Once);
        mockNotificationFactory.Verify(f => f.CreateEmailNotifier(), Times.Once);
    }
}
```

```
mockRepositoryFactory.Verify(f => f.CreateOrderRepository(), Times.Once);

// Verificar que produtos criados foram usados corretamente
mockPaymentProcessor.Verify(p => p.Process(It.IsAny<Payment>()), Times.Once);
mockNotifier.Verify(n => n.Send(It.IsAny<string>(), It.IsAny<string>()), Times.Once);
mockRepository.Verify(r => r.Save(order), Times.Once);
}
}
```

Compare este teste com o que seria necessário mockando o God Factory original. Aqui, cada mock é focado e específico. Não precisamos nos preocupar com dezenas de métodos que nunca serão chamados. O teste é claro, legível e mantível.

Benefícios Alcançados

Depois desta refatoração extensiva, vamos fazer uma pausa para analisar objetivamente os benefícios concretos que alcançamos. Não são apenas melhorias teóricas ou acadêmicas, mas vantagens práticas que você sentiria imediatamente no dia a dia de desenvolvimento.

Primeiro, responsabilidade única foi restaurada. Cada factory agora tem exatamente uma razão para mudar: quando novos tipos precisam ser adicionados ao seu domínio específico. NotificationFactory muda quando novos notificadores são adicionados. PaymentProcessorFactory muda quando novos processadores de pagamento são adicionados. As razões para mudança estão completamente isoladas.

Segundo, extensibilidade melhorou dramaticamente. Adicionar um novo tipo de notificador agora requer modificar apenas NotificationFactory e adicionar a nova classe de notificador. Nenhum outro código no sistema precisa mudar. Nenhum outro desenvolvedor precisa ser consultado. Não há risco de quebrar funcionalidade de pagamentos ao adicionar funcionalidade de notificações.

Terceiro, trabalho em equipe se tornou mais fluido. Cinco desenvolvedores podem agora trabalhar em cinco factories diferentes simultaneamente sem nenhum conflito de merge. Cada desenvolvedor trabalha em seu próprio arquivo, em seu próprio domínio, de forma completamente independente. A velocidade de desenvolvimento da equipe aumenta proporcionalmente.

Quarto, testabilidade aumentou em ordem de magnitude. Testes são agora focados, rápidos e mantíveis. Mockar INotificationFactory é trivial comparado a mockar o God Factory original. Testes falham apenas quando há problemas reais, não por mudanças em partes não relacionadas do sistema.

Quinto, o código se tornou autodocumentado. Quando você vê que OrderService depende de IPaymentProcessorFactory, INotificationFactory e IRepositoryFactory, você sabe imediatamente que este serviço trabalha com pagamentos, notificações e persistência de dados. A arquitetura comunica a intenção claramente.

Sexto, type safety foi restaurada. Não há mais strings mágicas que podem ser digitadas incorretamente. Não há mais casts perigosos que podem falhar. O compilador agora nos protege de muitos erros que antes só seriam descobertos em runtime ou pior, em produção.

Quando God Factory Pode Ser Aceitável

Embora tenhamos gasto tempo considerável demonstrando os problemas do God Factory, é importante reconhecer que contexto sempre importa em engenharia de software. Existem situações muito específicas onde um factory centralizado pode ser aceitável ou até mesmo pragmático.

Em protótipos descartáveis onde você está explorando uma ideia rapidamente e sabe que o código será reescrito completamente antes de ir para produção, a simplicidade de um factory centralizado pode acelerar a prototipagem. A chave aqui é realmente descartar ou refatorar completamente antes que o código entre em produção.

Em sistemas muito pequenos com menos de cinco ou seis tipos de objetos totais, onde toda a aplicação cabe confortavelmente em um único arquivo e será mantida por uma única pessoa, um factory simples e centralizado pode ser suficiente. Mas seja honesto sobre "muito pequeno" - a maioria dos sistemas cresce muito além deste ponto rapidamente.

Em ferramentas de linha de comando de uso único que executam uma tarefa específica e nunca evoluem, onde você sabe que nunca adicionará novos tipos de objetos, um factory centralizado simples pode ser aceitável. Novamente, a chave é certeza de que o código realmente nunca evoluirá.

Em exercícios acadêmicos focados em demonstrar um conceito específico não relacionado a factories, onde a factory é apenas infraestrutura de suporte e não o foco do aprendizado, simplicidade pode prevalecer. Mas mesmo em contextos educacionais, ensinar boas práticas desde o início geralmente é melhor.

A regra geral é: se você não tem certeza absoluta de que está em uma das situações excepcionais acima, evite God Factory. É muito mais fácil começar com factories focadas e especializadas desde o início do que refatorar um God Factory gigante depois que ele já cresceu descontroladamente.

CENÁRIO 2: O LOCALIZADOR ESCONDIDO (Service Locator Anti-Pattern)

O Código Problemático

Agora vamos examinar um anti-pattern mais sutil que frequentemente aparece quando desenvolvedores estão começando a aprender sobre dependency injection. Este padrão parece razoável à primeira vista e o código até funciona bem, mas esconde problemas sérios de design. Veja este código de um sistema de processamento de pedidos:

```
csharp
```

```

using Microsoft.Extensions.DependencyInjection;
using System;

// Factory que recebe IServiceProvider
public class PaymentFactory : IPaymentFactory
{
    private readonly IServiceProvider _serviceProvider;

    public PaymentFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public IPaymentProcessor Create(string type)
    {
        return type.ToLower() switch
        {
            "credit" => _serviceProvider.GetRequiredService<CreditCardProcessor>(),
            "boleto" => _serviceProvider.GetRequiredService<BoletoProcessor>(),
            "pix" => _serviceProvider.GetRequiredService<PixProcessor>(),
            _ => throw new NotSupportedException($"Tipo de pagamento não suportado: {type}")
        };
    }
}

// Serviço que parece usar DI corretamente
public class OrderService
{
    // Dependência escondida: usa IServiceProvider para resolver sob demanda
    private readonly IServiceProvider _serviceProvider;
    private readonly ILogger<OrderService> _logger;

    public OrderService(IServiceProvider serviceProvider, ILogger<OrderService> logger)
    {
        _serviceProvider = serviceProvider;
        _logger = logger;
    }

    public void ProcessOrder(Order order)
    {
        _logger.LogInformation($"Processando pedido {order.Id}");

        // Resolvendo dependências sob demanda ao invés de injetar no construtor
        var paymentFactory = _serviceProvider.GetRequiredService<IPaymentFactory>();
        var notificationService = _serviceProvider.GetRequiredService<INotificationService>();
        var orderRepository = _serviceProvider.GetRequiredService<IOrderRepository>();
    }
}

```

```

var validator = _serviceProvider.GetRequiredService<IOrderValidator>();

// Validar pedido
if (!validator.Validate(order))
{
    throw new InvalidOperationException("Pedido inválido");
}

// Processar pagamento
var processor = paymentFactory.Create(order.PaymentMethod);
var paymentResult = processor.Process(order.Payment);

if (!paymentResult.Success)
{
    throw new InvalidOperationException("Falha no processamento do pagamento");
}

// Salvar pedido
orderRepository.Save(order);

// Enviar notificação
notificationService.NotifyCustomer(order.CustomerId, "Pedido processado com sucesso");

_logger.LogInformation($"Pedido {order.Id} processado com sucesso");
}
}

// Outro serviço com o mesmo problema
public class ReportService
{
    private readonly IServiceProvider _serviceProvider;

    public ReportService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public void GenerateMonthlyReport()
    {
        // Resolvendo tudo sob demanda
        var orderRepository = _serviceProvider.GetRequiredService<IOrderRepository>();
        var reportFactory = _serviceProvider.GetRequiredService<IReportFactory>();
        var emailService = _serviceProvider.GetRequiredService<IEmailService>();

        var orders = orderRepository.GetOrdersByMonth(DateTime.Now.Month);
        var report = reportFactory.CreatePdfReport();
        report.Generate(orders);
    }
}

```

```
        emailService.SendReport(report);
    }
}

// Interfaces e classes de suporte
public interface IPaymentFactory
{
    IPaymentProcessor Create(string type);
}

public interface IPaymentProcessor
{
    PaymentResult Process(Payment payment);
}

public interface INotificationService
{
    void NotifyCustomer(string customerId, string message);
}

public interface IOrderRepository
{
    void Save(Order order);
    List<Order> GetOrdersByMonth(int month);
}

public interface IOrderValidator
{
    bool Validate(Order order);
}

public interface IReportFactory
{
    IReport CreatePdfReport();
}

public interface IReport
{
    void Generate(List<Order> orders);
}

public interface IEmailService
{
    void SendReport(IReport report);
}

public class Order
```

```

{
    public string Id { get; set; }
    public string PaymentMethod { get; set; }
    public string CustomerId { get; set; }
    public Payment Payment { get; set; }
}

public class Payment { }

public class PaymentResult
{
    public bool Success { get; set; }
}

public class CreditCardProcessor : IPaymentProcessor
{
    public PaymentResult Process(Payment payment)
    {
        return new PaymentResult { Success = true };
    }
}

public class BoletoProcessor : IPaymentProcessor
{
    public PaymentResult Process(Payment payment)
    {
        return new PaymentResult { Success = true };
    }
}

public class PixProcessor : IPaymentProcessor
{
    public PaymentResult Process(Payment payment)
    {
        return new PaymentResult { Success = true };
    }
}

```

Perguntas Investigativas

Antes de ler a análise, pare e reflita profundamente sobre estas questões. Elas foram desenhadas para guiar você a descobrir os problemas por conta própria, que é muito mais valioso do que simplesmente ler uma lista de problemas.

Primeiro, olhe para o construtor de OrderService. Quantas dependências ele explicitamente declara? Agora olhe para o método ProcessOrder. Quantas dependências ele realmente usa? Há uma discrepância entre essas duas

contagens? O que essa discrepância revela sobre a transparência do código?

Segundo, imagine que você está lendo o código de `OrderService` pela primeira vez tentando entender o que ele faz e de quais serviços ele depende. Você consegue saber isso olhando apenas para o construtor? Ou você precisa ler o corpo de cada método para descobrir todas as dependências? O que isso diz sobre a facilidade de compreensão do código?

Terceiro, pense sobre testes unitários. Se você quisesse testar `OrderService` em isolamento, como você configuraria seus mocks? Você consegue injetar mocks das dependências reais através do construtor? Como você mockaria `IServiceProvider` para retornar seus mocks? Quão complicado esse setup de teste seria?

Quarto, considere o princípio de Inversão de Dependência do SOLID. Este princípio diz que módulos de alto nível não devem depender de módulos de baixo nível, mas ambos devem depender de abstrações. `OrderService` depende de abstrações (`IPaymentFactory`, `INotificationService`, etc.) ou depende de um mecanismo de infraestrutura (`IServiceProvider`)? Qual é a diferença?

Quinto, analise a relação entre `OrderService` e `IServiceProvider`. Esta dependência é uma dependência de domínio (relacionada à lógica de negócio) ou uma dependência de infraestrutura (relacionada ao framework)? Serviços de domínio deveriam conhecer e depender de detalhes de infraestrutura do framework?

Sintomas Observáveis

O Service Locator anti-pattern apresenta sintomas característicos que, uma vez que você aprende a reconhecê-los, se tornam rapidamente identificáveis em qualquer base de código. Estes sintomas são mais sutis que os do God Factory, mas igualmente reveladores.

O primeiro sintoma é a presença ubíqua de `IServiceProvider` em construtores por toda a aplicação. Você começa a ver o mesmo padrão repetido em dezenas de classes: um construtor que recebe `IServiceProvider` e armazena em um campo privado. Quando você vê isso acontecendo sistematicamente, é um sinal claro de Service Locator.

O segundo sintoma é que construtores são enganosamente simples. Eles recebem apenas `IServiceProvider` e talvez um ou dois outros serviços óbvios como `ILogger`. Olhando para o construtor, você não tem ideia real de todas as dependências que a classe usa. As dependências verdadeiras estão escondidas dentro dos métodos, reveladas apenas através de chamadas a `GetRequiredService`.

O terceiro sintoma aparece quando você tenta entender uma classe. Você não consegue simplesmente olhar para o construtor e saber imediatamente de quais serviços a classe depende. Você precisa ler o código de cada método, procurando por chamadas a `GetRequiredService`, para montar uma imagem completa das dependências. Esta falta de transparência torna a compreensão do código significativamente mais difícil.

O quarto sintoma surge durante testes. Configurar testes para classes que usam Service Locator é surpreendentemente complicado. Você não pode simplesmente passar mocks através do construtor. Em vez disso, você precisa criar um mock de `IServiceProvider` e configurá-lo para retornar seus mocks quando `GetRequiredService` for chamado para tipos específicos. Este setup extra adiciona complexidade e fragilidade aos testes.

O quinto sintoma é que erros de configuração de DI aparecem em runtime, não em tempo de compilação. Se você esqueceu de registrar um serviço no container, você não descobre isso quando compila ou mesmo quando inicia a aplicação. Você descobre apenas quando o código específico que chama `GetRequiredService` para aquele tipo é executado. Isso pode ser em produção, em um caminho de código raramente executado.

Por Que É Problemático

Service Locator é problemático por razões que vão muito além de violações técnicas de princípios de design. Ele cria problemas práticos reais que afetam produtividade, qualidade e manutenibilidade do código diariamente.

O problema mais fundamental é que Service Locator esconde dependências. Quando você olha para o construtor de uma classe que usa Service Locator, você não sabe de quais serviços ela realmente depende. As dependências estão espalhadas pelo código dos métodos, reveladas apenas através de chamadas a `GetRequiredService`. Esta falta de transparência é uma violação direta do princípio de que dependências devem ser explícitas e óbvias.

Service Locator viola o princípio de Inversão de Dependência de uma forma sutil mas importante. Módulos de alto nível (como `OrderService` que implementa lógica de negócio) não deveriam depender de módulos de baixo nível (como mecanismos de infraestrutura do framework). Ambos deveriam depender de abstrações do domínio. Mas quando `OrderService` depende de `IServiceProvider`, ele está diretamente acoplado a um detalhe de implementação do framework de DI. Se você quisesse mudar de `Microsoft.Extensions.DependencyInjection` para outro container, você precisaria modificar `OrderService`.

A testabilidade sofre significativamente com Service Locator. Para testar uma classe que usa Service Locator, você precisa criar um mock complexo de `IServiceProvider` que sabe retornar os mocks corretos para cada tipo que pode ser solicitado. Isso não apenas adiciona código de setup aos testes, mas também torna os testes frágeis. Se você adicionar uma nova dependência em um método, todos os testes que exercitam aquele método precisam ter seus mocks de `IServiceProvider` atualizados.

Service Locator torna refatoração mais arriscada. Quando dependências estão explícitas no construtor, seu IDE pode ajudá-lo durante refatoração. Ele pode mostrar todos os lugares onde uma classe é instanciada, pode sugerir mudanças necessárias quando você adiciona um parâmetro ao construtor, pode até fazer algumas refatorações automaticamente. Com Service Locator, essas ferramentas não ajudam porque as dependências estão escondidas dentro do código dos métodos. Você está por conta própria para encontrar e atualizar todos os lugares relevantes.

Finalmente, Service Locator remove a ajuda do compilador. Com Constructor Injection adequado, se você esqueceu de registrar um serviço no container, você descobre isso imediatamente quando tenta construir o objeto raiz da aplicação, tipicamente durante startup. Com Service Locator, você só descobre quando o código que chama `GetRequiredService` é executado, que pode ser muito depois do startup, em um caminho de código raramente executado, possivelmente até em produção.

A Refatoração: Eliminando Service Locator

Vamos transformar este código que usa Service Locator em código que usa Constructor Injection apropriadamente. Esta refatoração é mais direta que a do God Factory, mas requer disciplina para aplicar

consistentemente em toda a base de código.

Passo 1: Identificar Dependências Reais

O primeiro passo é identificar todas as dependências reais de cada classe que está usando Service Locator. Vamos começar com OrderService. Lendo o método ProcessOrder, vemos que ele usa IPaymentFactory, INotificationService, IOrderRepository e IOrderValidator. Estas são as dependências reais, não IServiceProvider.

Passo 2: Mover Dependências para o Construtor

Agora movemos todas essas dependências reais para o construtor onde elas pertencem.

```
csharp
```


// ANTES: Service Locator esconde dependências

```
public class OrderService
{
    private readonly IServiceProvider _serviceProvider;
    private readonly ILogger<OrderService> _logger;

    public OrderService(IServiceProvider serviceProvider, ILogger<OrderService> logger)
    {
        _serviceProvider = serviceProvider;
        _logger = logger;
    }

    public void ProcessOrder(Order order)
    {
        // Resolvendo sob demanda
        var paymentFactory = _serviceProvider.GetRequiredService<IPaymentFactory>();
        var notificationService = _serviceProvider.GetRequiredService<INotificationService>();
        var orderRepository = _serviceProvider.GetRequiredService<IOrderRepository>();
        var validator = _serviceProvider.GetRequiredService<IOrderValidator>();

        // ... usar serviços
    }
}
```

// DEPOIS: Constructor Injection torna dependências explícitas

```
public class OrderService
{
    // Todas as dependências são campos privados
    private readonly IPaymentFactory _paymentFactory;
    private readonly INotificationService _notificationService;
    private readonly IOrderRepository _orderRepository;
    private readonly IOrderValidator _validator;
    private readonly ILogger<OrderService> _logger;

    // Construtor declara todas as dependências explicitamente
    public OrderService(
        IPaymentFactory paymentFactory,
        INotificationService notificationService,
        IOrderRepository orderRepository,
        IOrderValidator validator,
        ILogger<OrderService> logger)
    {
        // Validação de null para todas as dependências
        _paymentFactory = paymentFactory ?? throw new ArgumentNullException(nameof(paymentFactory));
        _notificationService = notificationService ?? throw new ArgumentNullException(nameof(notificationService));
        _orderRepository = orderRepository ?? throw new ArgumentNullException(nameof(orderRepository));
    }
}
```

```

    _validator = validator ?? throw new ArgumentNullException(nameof(validator));
    _logger = logger ?? throw new ArgumentNullException(nameof(logger));
}

public void ProcessOrder(Order order)
{
    _logger.LogInformation($"Processando pedido {order.Id}");

    // Usar dependências já injetadas - sem resolução dinâmica
    if (!_validator.Validate(order))
    {
        throw new InvalidOperationException("Pedido inválido");
    }

    var processor = _paymentFactory.Create(order.PaymentMethod);
    var paymentResult = processor.Process(order.Payment);

    if (!paymentResult.Success)
    {
        throw new InvalidOperationException("Falha no processamento do pagamento");
    }

    _orderRepository.Save(order);
    _notificationService.NotifyCustomer(order.CustomerId, "Pedido processado com sucesso");

    _logger.LogInformation($"Pedido {order.Id} processado com sucesso");
}
}

```

Veja a diferença dramática. Agora, apenas olhando para o construtor, você sabe exatamente de quais serviços OrderService depende. Não há surpresas escondidas. Não há dependência de IServiceProvider. O serviço depende de abstrações do domínio, não de infraestrutura de framework.

Passo 3: Aplicar o Mesmo Padrão a Outras Classes

Aplicamos a mesma transformação a ReportService e qualquer outra classe que estava usando Service Locator.

csharp

// ANTES: Service Locator

```
public class ReportService
{
    private readonly IServiceProvider _serviceProvider;

    public ReportService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public void GenerateMonthlyReport()
    {
        var orderRepository = _serviceProvider.GetRequiredService<IOrderRepository>();
        var reportFactory = _serviceProvider.GetRequiredService<IReportFactory>();
        var emailService = _serviceProvider.GetRequiredService<IEmailService>();

        // ... usar serviços
    }
}
```

// DEPOIS: Constructor Injection

```
public class ReportService
{
    private readonly IOrderRepository _orderRepository;
    private readonly IReportFactory _reportFactory;
    private readonly IEmailService _emailService;

    public ReportService(
        IOrderRepository orderRepository,
        IReportFactory reportFactory,
        IEmailService emailService)
    {
        _orderRepository = orderRepository ?? throw new ArgumentNullException(nameof(orderRepository));
        _reportFactory = reportFactory ?? throw new ArgumentNullException(nameof(reportFactory));
        _emailService = emailService ?? throw new ArgumentNullException(nameof(emailService));
    }

    public void GenerateMonthlyReport()
    {
        // Usar campos injetados diretamente
        var orders = _orderRepository.GetOrdersByMonth(DateTime.Now.Month);
        var report = _reportFactory.CreatePdfReport();
        report.Generate(orders);
        _emailService.SendReport(report);
    }
}
```

Passo 4: Exceção - Quando IServiceProvider É Justificado

É importante notar que há um lugar onde IServiceProvider é perfeitamente apropriado e aceitável: dentro de factories. Vamos ver porque PaymentFactory usando IServiceProvider é correto.

```
csharp

// CORRETO: Factory pode usar IServiceProvider
public class PaymentFactory : IPaymentFactory
{
    private readonly IServiceProvider _serviceProvider;

    // Factory precisa resolver tipos dinamicamente em runtime
    public PaymentFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider ?? throw new ArgumentNullException(nameof(serviceProvider));
    }

    public IPaymentProcessor Create(string type)
    {
        // Factory é essencialmente um ponto de extensibilidade
        // que precisa resolver tipos dinamicamente
        return type.ToLower() switch
        {
            "credit" => _serviceProvider.GetRequiredService<CreditCardProcessor>(),
            "boleto" => _serviceProvider.GetRequiredService<BoletoProcessor>(),
            "pix" => _serviceProvider.GetRequiredService<PixProcessor>(),
            _ => throw new NotSupportedException($"Tipo não suportado: {type}")
        };
    }
}
```

Por que isso é aceitável em uma factory mas não em OrderService? Porque o propósito fundamental de uma factory é criar objetos dinamicamente baseado em informações de runtime. Você não sabe em tempo de compilação qual tipo específico de IPaymentProcessor será necessário. Isso é determinado em runtime baseado em order.PaymentMethod. Factory é essencialmente um ponto de extensibilidade que traduz entre um identificador runtime (string, enum, etc.) e um tipo concreto resolvido do container.

OrderService, por outro lado, sempre usa as mesmas dependências. Não há nada dinâmico sobre quais serviços ele precisa. Ele sempre precisa de IPaymentFactory, INotificationService, IOrderRepository e IOrderValidator. Estas dependências são conhecidas em tempo de design e devem ser explícitas no construtor.

Passo 5: Melhorar Testes

Agora vamos ver como os testes ficam muito mais simples e claros com Constructor Injection apropriado.

csharp

```
using Xunit;
using Moq;

public class OrderServiceTests
{
    [Fact]
    public void ProcessOrder_WithValidOrder_ProcessesSuccessfully()
    {
        // Arrange: Criar mocks de todas as dependências
        var mockPaymentFactory = new Mock<IPaymentFactory>();
        var mockNotificationService = new Mock<INotificationService>();
        var mockOrderRepository = new Mock<IOrderRepository>();
        var mockValidator = new Mock<IOrderValidator>();
        var mockLogger = new Mock<ILogger<OrderService>>();

        // Configurar comportamento dos mocks
        mockValidator.Setup(v => v.Validate(It.IsAny<Order>())).Returns(true);

        var mockProcessor = new Mock<IPaymentProcessor>();
        mockProcessor
            .Setup(p => p.Process(It.IsAny<Payment>()))
            .Returns(new PaymentResult { Success = true });

        mockPaymentFactory
            .Setup(f => f.Create(It.IsAny<string>()))
            .Returns(mockProcessor.Object);

        // Injetar mocks diretamente no construtor - simples e direto
        var service = new OrderService(
            mockPaymentFactory.Object,
            mockNotificationService.Object,
            mockOrderRepository.Object,
            mockValidator.Object,
            mockLogger.Object
        );

        var order = new Order
        {
            Id = "123",
            PaymentMethod = "credit",
            CustomerId = "customer1",
            Payment = new Payment()
        };

        // Act
        service.ProcessOrder(order);
    }
}
```

```

// Assert: Verificar que todos os serviços foram usados corretamente
mockValidator.Verify(v => v.Validate(order), Times.Once);
mockPaymentFactory.Verify(f => f.Create("credit"), Times.Once);
mockProcessor.Verify(p => p.Process(order.Payment), Times.Once);
mockOrderRepository.Verify(r => r.Save(order), Times.Once);
mockNotificationService.Verify(
    n => n.NotifyCustomer("customer1", It.IsAny<string>()),
    Times.Once
);
}

```

[Fact]

```

public void ProcessOrder_WithInvalidOrder_ThrowsException()
{
    // Arrange
    var mockPaymentFactory = new Mock<IPaymentFactory>();
    var mockNotificationService = new Mock<INotificationService>();
    var mockOrderRepository = new Mock<IOrderRepository>();
    var mockValidator = new Mock<IOrderValidator>();
    var mockLogger = new Mock<ILogger<OrderService>>();

    // Validator retorna false para simular pedido inválido
    mockValidator.Setup(v => v.Validate(It.IsAny<Order>())).Returns(false);

    var service = new OrderService(
        mockPaymentFactory.Object,
        mockNotificationService.Object,
        mockOrderRepository.Object,
        mockValidator.Object,
        mockLogger.Object
    );

    var invalidOrder = new Order { Id = "invalid" };

    // Act & Assert
    var exception = Assert.Throws<InvalidOperationException>(() =>
    {
        service.ProcessOrder(invalidOrder);
    });

    Assert.Equal("Pedido inválido", exception.Message);

    // Verificar que serviços posteriores não foram chamados
    mockPaymentFactory.Verify(f => f.Create(It.IsAny<string>()), Times.Never);
    mockOrderRepository.Verify(r => r.Save(It.IsAny<Order>()), Times.Never);
}

```

```
}  
}
```

Compare estes testes com o que seria necessário se OrderService usasse Service Locator. Aqui, configurar o teste é direto e intuitivo. Criamos mocks das dependências e as injetamos através do construtor. Não há necessidade de mockar IServiceProvider. Não há configuração complexa de qual mock retornar para qual tipo. Tudo é explícito, claro e type-safe.

Benefícios Alcançados

A eliminação do Service Locator anti-pattern traz benefícios imediatos e tangíveis que você sentiria desde o primeiro dia após a refatoração.

Primeiro, transparência foi restaurada. Agora, qualquer pessoa pode olhar para o construtor de OrderService e saber imediatamente de quais serviços ele depende. Não há dependências escondidas. Não há surpresas. O código comunica sua intenção claramente apenas através de sua assinatura.

Segundo, compile-time safety foi recuperada. Se você esquecer de registrar um serviço no container, você descobre isso imediatamente ao tentar construir a aplicação, tipicamente durante startup. Você não precisa esperar até que um caminho específico de código seja executado para descobrir o problema. Erros de configuração são detectados cedo, quando são mais baratos de consertar.

Terceiro, testabilidade melhorou dramaticamente. Testes agora são simples e diretos. Você cria mocks das dependências, injeta através do construtor, e pronto. Não há configuração complexa de container. Não há mocking de IServiceProvider. Os testes são mais fáceis de escrever, mais fáceis de entender e mais robustos.

Quarto, refatoração se tornou mais segura e mais fácil. Quando você adiciona uma nova dependência, você a adiciona ao construtor. O compilador imediatamente mostra todos os lugares onde a classe é instanciada que agora precisam fornecer a nova dependência. Seu IDE pode até fazer algumas dessas mudanças automaticamente. Com Service Locator, você estaria por conta própria procurando por todos os lugares relevantes manualmente.

Quinto, o código respeita Inversão de Dependência apropriadamente. OrderService agora depende apenas de abstrações do domínio de negócio (IPaymentFactory, INotificationService, etc.), não de detalhes de infraestrutura (IServiceProvider). Isso significa que a lógica de negócio está isolada de detalhes de implementação do framework, tornando o código mais portátil e mais testável.

Quando Service Locator Pode Ser Aceitável

Embora tenhamos dedicado esforço considerável a demonstrar os problemas do Service Locator, honestidade intelectual requer que reconheçamos as raras situações onde seu uso pode ser justificado.

O uso mais legítimo de IServiceProvider é em factories, como já discutimos. Factories são pontos de extensibilidade que precisam resolver tipos dinamicamente em runtime baseado em informações que não são conhecidas em tempo de design. Neste contexto específico, usar IServiceProvider é apropriado e aceitável.

Outro uso legítimo é em frameworks e bibliotecas de infraestrutura que fornecem pontos de extensão para código de aplicação. Por exemplo, um framework de plugins pode usar `IServiceProvider` internamente para resolver plugins dinamicamente. Mas note que isso é código de framework, não código de aplicação. Seu código de domínio de negócio geralmente não deveria usar este padrão.

Um terceiro caso é durante migração gradual de código legacy. Se você tem uma base de código muito grande que não usa DI de forma alguma e está gradualmente introduzindo DI, você pode temporariamente usar `Service Locator` como passo intermediário. Mas isso deve ser reconhecido explicitamente como débito técnico temporário, e deve haver um plano claro para eventualmente refatorar para `Constructor Injection` apropriado.

A regra geral é: se você está escrevendo código de aplicação (lógica de negócio, serviços de domínio, controladores, etc.), use `Constructor Injection`. Reserve `IServiceProvider` apenas para casos excepcionais onde você genuinamente precisa resolver tipos dinamicamente em runtime, e mesmo assim, isole esse uso em factories ou componentes de infraestrutura dedicados.

CENÁRIO 3: FACTORY COM AMNÉSIA (Stateful Factory Anti-Pattern)

O Código Problemático

Este anti-pattern é particularmente insidioso porque o código parece funcionar perfeitamente em desenvolvimento e testes simples, mas pode causar bugs sutis e difíceis de reproduzir em produção quando múltiplas threads estão envolvidas. Vamos examinar um factory que mantém estado mutável:

```
csharp
```

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Threading;

// Factory que mantém estado mutável entre criações
public class ReportGeneratorFactory : IReportGeneratorFactory
{
    // Estado mutável compartilhado - PROBLEMA!
    private int _reportCounter = 0;
    private readonly Dictionary<string, DateTime> _lastGenerationTime;
    private readonly Dictionary<string, int> _generationCount;
    private string _lastFormat = null;

    // Cache que pode causar memory leaks
    private readonly Dictionary<string, IReportGenerator> _cache;

    public ReportGeneratorFactory()
    {
        _lastGenerationTime = new Dictionary<string, DateTime>();
        _generationCount = new Dictionary<string, int>();
        _cache = new Dictionary<string, IReportGenerator>();
    }

    public IReportGenerator Create(string format, ReportOptions options = null)
    {
        // Incrementando contador compartilhado - não é thread-safe!
        _reportCounter++;

        // Registrando último formato usado - estado global mutável
        _lastFormat = format;

        // Registrando tempo de geração - compartilhado entre todas as requisições
        _lastGenerationTime[format] = DateTime.Now;

        // Contador de gerações por formato - também compartilhado
        if (_generationCount.ContainsKey(format))
        {
            _generationCount[format]++;
        }
        else
        {
            _generationCount[format] = 1;
        }

        // Cache de generators - pode causar problemas se generators não são thread-safe
    }
}
```

```
if (_cache.ContainsKey(format))
{
    Console.WriteLine($"Retornando generator do cache para formato {format}");
    return _cache[format];
}

IReportGenerator generator = format.ToLower() switch
{
    "pdf" => new PdfReportGenerator(_reportCounter),
    "excel" => new ExcelReportGenerator(_reportCounter),
    "html" => new HtmlReportGenerator(_reportCounter),
    _ => throw new NotSupportedException($"Formato não suportado: {format} ")
};

// Armazenando no cache
_cache[format] = generator;

return generator;
}

// Métodos que expõem estado interno
public int GetTotalReportsGenerated()
{
    return _reportCounter;
}

public string GetLastFormatUsed()
{
    return _lastFormat;
}

public DateTime GetLastGenerationTime(string format)
{
    return _lastGenerationTime.ContainsKey(format)
        ? _lastGenerationTime[format]
        : DateTime.MinValue;
}

public int GetGenerationCount(string format)
{
    return _generationCount.ContainsKey(format)
        ? _generationCount[format]
        : 0;
}
}

// Report generators que recebem estado do factory
```

```
public class PdfReportGenerator : IReportGenerator
{
    private readonly int _reportNumber;

    public PdfReportGenerator(int reportNumber)
    {
        _reportNumber = reportNumber;
    }

    public void Generate(ReportData data)
    {
        Console.WriteLine($"[PDF] Gerando relatório #{_reportNumber}");
        // Simular geração
        Thread.Sleep(100);
    }
}

public class ExcelReportGenerator : IReportGenerator
{
    private readonly int _reportNumber;

    public ExcelReportGenerator(int reportNumber)
    {
        _reportNumber = reportNumber;
    }

    public void Generate(ReportData data)
    {
        Console.WriteLine($"[Excel] Gerando relatório #{_reportNumber}");
        Thread.Sleep(100);
    }
}

public class HtmlReportGenerator : IReportGenerator
{
    private readonly int _reportNumber;

    public HtmlReportGenerator(int reportNumber)
    {
        _reportNumber = reportNumber;
    }

    public void Generate(ReportData data)
    {
        Console.WriteLine($"[HTML] Gerando relatório #{_reportNumber}");
        Thread.Sleep(100);
    }
}
```

```

}

// Código cliente que usa o factory em ambiente multi-thread
public class ReportService
{
    private readonly IReportGeneratorFactory _factory;

    public ReportService(IReportGeneratorFactory factory)
    {
        _factory = factory;
    }

    public void GenerateReport(string format, ReportData data)
    {
        var generator = _factory.Create(format);
        generator.Generate(data);

        // Código cliente pode estar dependendo do estado do factory
        Console.WriteLine($"Total de relatórios gerados até agora: {_factory.GetTotalReportsGenerated()}");
        Console.WriteLine($"Último formato usado: {_factory.GetLastFormatUsed()}");
    }
}

// Interfaces e classes de suporte
public interface IReportGeneratorFactory
{
    IReportGenerator Create(string format, ReportOptions options = null);
    int GetTotalReportsGenerated();
    string GetLastFormatUsed();
    DateTime GetLastGenerationTime(string format);
    int GetGenerationCount(string format);
}

public interface IReportGenerator
{
    void Generate(ReportData data);
}

public class ReportOptions { }
public class ReportData { }

// Programa de demonstração que mostra o problema em ação
class Program
{
    static void Main()
    {
        var factory = new ReportGeneratorFactory();
    }
}

```

```

var service = new ReportService(factory);

// Simular múltiplas threads gerando relatórios simultaneamente
var threads = new List<Thread>();

for (int i = 0; i < 10; i++)
{
    int threadNumber = i;
    var thread = new Thread(() =>
    {
        string format = threadNumber % 2 == 0 ? "pdf" : "excel";
        service.GenerateReport(format, new ReportData());
    });
    threads.Add(thread);
    thread.Start();
}

// Aguardar todas as threads
foreach (var thread in threads)
{
    thread.Join();
}

Console.WriteLine($"Total final de relatórios: {factory.GetTotalReportsGenerated()}");
Console.WriteLine($"Esperado: 10, mas pode ser diferente devido a race conditions!");
}
}

```

Perguntas Investigativas

Estas perguntas foram cuidadosamente elaboradas para guiá-lo através de uma análise profunda dos problemas deste código. Reserve tempo para pensar honestamente sobre cada uma antes de continuar.

Primeiro, identifique todo estado mutável na classe ReportGeneratorFactory. Para cada pedaço de estado, pergunte-se: este estado é compartilhado entre todas as chamadas a Create? Se múltiplas threads chamarem Create simultaneamente, este estado poderia ficar em um estado inconsistente? Anote cada variável de instância e analise seu uso.

Segundo, execute mentalmente este cenário: Thread A chama Create("pdf") e Thread B chama Create("excel") exatamente ao mesmo tempo. Desenhe uma linha do tempo mostrando como `_reportCounter` seria incrementado por ambas as threads. É possível que ambas leiam o mesmo valor, incrementem, e escrevam de volta? O que aconteceria com o valor final do contador?

Terceiro, considere o cache implementado em `_cache`. Se um IReportGenerator é retornado do cache e usado por múltiplas threads simultaneamente, que problemas poderiam surgir? Os generators (PdfReportGenerator,

ExcelReportGenerator) são thread-safe? Se não forem, o que aconteceria se duas threads tentassem usar o mesmo generator do cache ao mesmo tempo?

Quarto, analise os métodos que expõem estado interno como `GetLastFormatUsed()` e `GetTotalReportsGenerated()`. Se você chamar esses métodos, quanto você pode confiar nos valores retornados? Eles representam uma visão consistente do estado? Poderiam mudar entre chamadas sucessivas em um ambiente multi-thread?

Quinto, pense sobre testabilidade. Se você quisesse testar que o cache está funcionando corretamente, como você testaria isso? O comportamento do factory muda baseado em estado anterior? Testes executados em ordem diferente produziram resultados diferentes? O que isso implica sobre a determinismo e confiabilidade dos testes?

Sintomas Observáveis

Stateful Factory anti-pattern apresenta sintomas que podem ser sutis em desenvolvimento mas se tornam óbvios em produção, especialmente sob carga. Reconhecer estes sintomas precocemente pode prevenir bugs sérios.

O primeiro sintoma é comportamento diferente entre ambientes. O código funciona perfeitamente em desenvolvimento local onde há apenas uma thread executando. Testes unitários passam consistentemente. Mas em produção, sob carga real com múltiplas requisições simultâneas, bugs intermitentes aparecem. Relatórios às vezes têm números duplicados. Contadores às vezes pulam valores. O comportamento não é determinístico.

O segundo sintoma são race conditions manifestando como bugs intermitentes impossíveis de reproduzir consistentemente. Você recebe um bug report dizendo que dois relatórios receberam o mesmo número de identificação. Você tenta reproduzir localmente mas não consegue. Você adiciona logging e o problema desaparece (porque o logging muda o timing). Estes são sintomas clássicos de problemas de concorrência relacionados a estado compartilhado mutável.

O terceiro sintoma é que testes são inconsistentes ou dependem de ordem de execução. Se você executar testes individualmente, eles passam. Se você executar toda a suíte de testes, alguns falham aleatoriamente. Se você executar os mesmos testes em ordem diferente, resultados diferentes aparecem. Esta dependência de ordem e estado anterior é um sinal claro de estado global mutável problemático.

O quarto sintoma aparece em logs e monitoramento. Você vê contador de relatórios gerados não batendo com número real de relatórios no sistema. Você vê valores de "último formato usado" que não fazem sentido baseado nas requisições realmente processadas. Métricas e estatísticas parecem incorretas ou inconsistentes.

O quinto sintoma é memory leaks sutis. O cache nunca libera objetos. Cada formato de relatório solicitado adiciona uma entrada ao cache que nunca é removida. Em um sistema de longa duração, a memória do factory cresce indefinidamente. Eventualmente, você pode até ter out of memory exceptions, especialmente se os generators em cache mantêm recursos significativos.

Por Que É Problemático

Stateful Factory é problemático por múltiplas razões técnicas e práticas que vão desde problemas de correteude até dificuldades de manutenção e teste.

O problema mais sério é lack of thread-safety. Em aplicações web modernas, que são inerentemente multi-threaded, múltiplas requisições simultâneas podem chamar o factory ao mesmo tempo. Quando múltiplas threads acessam e modificam o mesmo estado mutável sem sincronização apropriada, race conditions ocorrem. O contador `_reportCounter` é incrementado por múltiplas threads, mas a operação `_reportCounter++` não é atômica. É uma sequência de ler o valor, incrementar e escrever de volta. Duas threads podem ler o mesmo valor, ambas incrementar, e ambas escrever de volta, resultando em apenas um incremento ao invés de dois.

O segundo problema é que state mutável compartilhado viola o princípio de que factories devem ser stateless. Uma factory tem uma responsabilidade clara e única: criar objetos. Ela não deveria manter informação sobre quantos objetos criou, quando os criou, ou em que ordem os criou. Estas responsabilidades de tracking e estatísticas pertencem a componentes separados dedicados a essas preocupações.

O terceiro problema é que o cache implementado desta forma pode causar bugs sutis. Se os generators não forem thread-safe (e a maioria não é por padrão), retornar a mesma instância de um generator para múltiplas threads cria problemas. Se uma thread está usando o generator enquanto outra thread também tenta usá-lo, o comportamento resultante é indefinido e pode variar de exceptions até corrupção silenciosa de dados.

O quarto problema é testabilidade severamente comprometida. Testes que dependem de um factory stateful não são isolados ou independentes. O estado de um teste vaza para outros testes. Se você executar Teste A seguido por Teste B, Teste B vê estado deixado por Teste A. Se você executar testes em paralelo (como muitos test runners modernos fazem por padrão), testes interferem uns com os outros de formas imprevisíveis.

O quinto problema é que métodos que expõem estado interno (`GetTotalReportsGenerated`, `GetLastFormatUsed`) criam uma API enganosa. Em um ambiente single-threaded, estes métodos parecem úteis. Mas em um ambiente multi-threaded, os valores que eles retornam são essencialmente inúteis porque podem mudar a qualquer momento. Entre o momento que você chama `GetTotalReportsGenerated()` e o momento que você usa o valor retornado, outras threads podem ter criado mais relatórios, tornando o valor que você obteve obsoleto imediatamente.

A Refatoração: Tornando Factory Stateless

Vamos transformar este factory stateful problemático em um factory stateless correto e thread-safe. Esta refatoração envolve não apenas mudar o código do factory, mas também repensar onde responsabilidades de tracking e estatísticas realmente pertencem.

Passo 1: Remover Todo Estado Mutável

O primeiro passo, e o mais importante, é remover completamente todo estado mutável do factory. Uma factory deve ser stateless. Ela não mantém informação entre chamadas.

csharp

// DEPOIS: Factory completamente stateless

```
public class ReportGeneratorFactory : IReportGeneratorFactory
{
    // Sem campos de instância mutáveis - apenas dependências imutáveis se necessário
    private readonly IServiceProvider _serviceProvider;

    public ReportGeneratorFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public IReportGenerator Create(string format, ReportOptions options = null)
    {
        // Criação pura sem efeitos colaterais em estado compartilhado
        return format.ToLower() switch
        {
            "pdf" => _serviceProvider.GetRequiredService<PdfReportGenerator>(),
            "excel" => _serviceProvider.GetRequiredService<ExcelReportGenerator>(),
            "html" => _serviceProvider.GetRequiredService<HtmlReportGenerator>(),
            _ => throw new NotSupportedException($"Formato não suportado: {format}")
        };
    }
}
```

Veja como o factory agora é simples e focado. Ele tem apenas uma responsabilidade: mapear um formato string para uma implementação concreta de `IReportGenerator`. Não há contadores. Não há timestamps. Não há cache. Não há estado mutável de qualquer tipo. Isto significa que o factory é automaticamente thread-safe porque não há estado para se tornar inconsistente.

Passo 2: Mover Responsabilidades para Componentes Apropriados

As responsabilidades de tracking (contar relatórios gerados, registrar timestamps, etc.) não desaparecem - elas simplesmente se movem para componentes dedicados que são projetados especificamente para essas preocupações.

csharp

// Componente dedicado para tracking de métricas

public interface IReportMetrics

```
{  
    void RecordReportGeneration(string format);  
    int GetTotalReportsGenerated();  
    int GetGenerationCount(string format);  
    DateTime GetLastGenerationTime(string format);  
}
```

// Implementação thread-safe usando coleções concorrentes

public class ReportMetrics : IReportMetrics

```
{  
    // ConcurrentDictionary é thread-safe  
    private readonly ConcurrentDictionary<string, int> _generationCount;  
    private readonly ConcurrentDictionary<string, DateTime> _lastGenerationTime;
```

// AtomicInteger para contador total

```
    private int _totalCount;
```

public ReportMetrics()

```
{  
    _generationCount = new ConcurrentDictionary<string, int>();  
    _lastGenerationTime = new ConcurrentDictionary<string, DateTime>();  
    _totalCount = 0;  
}
```

public void RecordReportGeneration(string format)

```
{  
    // Incremento atômico - thread-safe  
    Interlocked.Increment(ref _totalCount);  
  
    // AddOrUpdate é atômico em ConcurrentDictionary  
    _generationCount.AddOrUpdate(format, 1, (key, oldValue) => oldValue + 1);  
  
    // TryUpdate garante atomicidade  
    _lastGenerationTime[format] = DateTime.Now;  
}
```

public int GetTotalReportsGenerated()

```
{  
    // Leitura de int é atômica em .NET  
    return _totalCount;  
}
```

public int GetGenerationCount(string format)

```
{
```

```
        return _generationCount.TryGetValue(format, out var count) ? count : 0;
    }

    public DateTime GetLastGenerationTime(string format)
    {
        return _lastGenerationTime.TryGetValue(format, out var time)
            ? time
            : DateTime.MinValue;
    }
}
```

Agora temos um componente dedicado para métricas que é explicitamente thread-safe usando primitivas de concorrência apropriadas do .NET como Interlocked e ConcurrentDictionary. Este componente tem uma responsabilidade clara: tracking de métricas de geração de relatórios.

Passo 3: Refatorar Código Cliente

O código cliente agora usa tanto o factory (para criação) quanto o componente de métricas (para tracking) como dependências separadas.

```
csharp
```

```
public class ReportService
{
    private readonly IReportGeneratorFactory _factory;
    private readonly IReportMetrics _metrics;
    private readonly ILogger<ReportService> _logger;

    // Constructor injection de ambas as dependências
    public ReportService(
        IReportGeneratorFactory factory,
        IReportMetrics metrics,
        ILogger<ReportService> logger)
    {
        _factory = factory ?? throw new ArgumentNullException(nameof(factory));
        _metrics = metrics ?? throw new ArgumentNullException(nameof(metrics));
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
    }

    public void GenerateReport(string format, ReportData data)
    {
        try
        {
            // Criar generator usando factory stateless
            var generator = _factory.Create(format);

            // Gerar relatório
            generator.Generate(data);

            // Registrar métrica usando componente dedicado
            _metrics.RecordReportGeneration(format);

            _logger.LogInformation($"Relatório {format} gerado com sucesso");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"Erro ao gerar relatório {format}");
            throw;
        }
    }

    public ReportStatistics GetStatistics()
    {
        return new ReportStatistics
        {
            TotalGenerated = _metrics.GetTotalReportsGenerated(),
            PdfCount = _metrics.GetGenerationCount("pdf"),
            ExcelCount = _metrics.GetGenerationCount("excel"),
        }
    }
}
```

```
        HtmlCount = _metrics.GetGenerationCount("html")
    };
}
}

public class ReportStatistics
{
    public int TotalGenerated { get; set; }
    public int PdfCount { get; set; }
    public int ExcelCount { get; set; }
    public int HtmlCount { get; set; }
}
```

Veja como agora há separação clara de responsabilidades. ReportService usa IReportGeneratorFactory para criar generators e usa IReportMetrics para tracking. Cada dependência tem seu propósito específico e bem definido.

Passo 4: Abordar Caching Se Realmente Necessário

Se você genuinamente precisa de caching de generators por razões de performance (o que é raro), isso deve ser implementado de forma thread-safe e consciente. Mas primeiro, questione se você realmente precisa de cache. Criar um novo generator cada vez geralmente é rápido o suficiente, especialmente quando resolvido de um container DI.

Se você realmente precisa de cache:

```
csharp
```

```
// Cache thread-safe usando Lazy<T> e ConcurrentDictionary
public class CachingReportGeneratorFactory : IReportGeneratorFactory
{
    private readonly IServiceProvider _serviceProvider;
    private readonly ConcurrentDictionary<string, Lazy<IReportGenerator>> _cache;

    public CachingReportGeneratorFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
        _cache = new ConcurrentDictionary<string, Lazy<IReportGenerator>>();
    }

    public IReportGenerator Create(string format, ReportOptions options = null)
    {
        // GetOrAdd com Lazy<T> garante que apenas uma thread cria cada generator
        var lazyGenerator = _cache.GetOrAdd(format, key =>
        {
            return new Lazy<IReportGenerator>(() => CreateGenerator(key));
        });

        return lazyGenerator.Value;
    }

    private IReportGenerator CreateGenerator(string format)
    {
        return format.ToLower() switch
        {
            "pdf" => _serviceProvider.GetRequiredService<PdfReportGenerator>(),
            "excel" => _serviceProvider.GetRequiredService<ExcelReportGenerator>(),
            "html" => _serviceProvider.GetRequiredService<HtmlReportGenerator>(),
            _ => throw new NotSupportedException($"Formato não suportado: {format}")
        };
    }
}
```

Este padrão usando `Lazy<T>` garante que mesmo com múltiplas threads solicitando o mesmo formato simultaneamente, apenas uma thread criará o generator, e todas as outras esperarão e receberão a mesma instância. Mas note: isto só é seguro se os generators em si forem stateless e thread-safe!

Passo 5: Registro no Container DI

csharp

```
public void ConfigureServices(IServiceCollection services)
{
    // Registrar generators como Transient (nova instância cada vez)
    services.AddTransient<PdfReportGenerator>();
    services.AddTransient<ExcelReportGenerator>();
    services.AddTransient<HtmlReportGenerator>();

    // Factory stateless como Singleton - é seguro porque não há estado
    services.AddSingleton<IReportGeneratorFactory, ReportGeneratorFactory>();

    // Se precisa de cache, use implementação separada
    // services.AddSingleton<IReportGeneratorFactory, CachingReportGeneratorFactory>();

    // Metrics como Singleton - é thread-safe por design
    services.AddSingleton<IReportMetrics, ReportMetrics>();

    // Service usa ambas as dependências
    services.AddScoped<ReportService>();
}
```

Passo 6: Melhorar Testes

Testes agora são determinísticos e isolados porque o factory não mantém estado.

csharp

```
public class ReportServiceTests
{
    [Fact]
    public void GenerateReport_RecordsMetricsCorrectly()
    {
        // Arrange
        var mockFactory = new Mock<IReportGeneratorFactory>();
        var mockMetrics = new Mock<IReportMetrics>();
        var mockLogger = new Mock<ILogger<ReportService>>();

        var mockGenerator = new Mock<IReportGenerator>();
        mockFactory
            .Setup(f => f.Create("pdf", null))
            .Returns(mockGenerator.Object);

        var service = new ReportService(
            mockFactory.Object,
            mockMetrics.Object,
            mockLogger.Object
        );

        // Act
        service.GenerateReport("pdf", new ReportData());

        // Assert
        mockFactory.Verify(f => f.Create("pdf", null), Times.Once);
        mockGenerator.Verify(g => g.Generate(It.IsAny<ReportData>()), Times.Once);
        mockMetrics.Verify(m => m.RecordReportGeneration("pdf"), Times.Once);
    }

    [Fact]
    public void MultipleReports_EachGetsNewGenerator()
    {
        // Arrange
        var mockFactory = new Mock<IReportGeneratorFactory>();
        var mockMetrics = new Mock<IReportMetrics>();
        var mockLogger = new Mock<ILogger<ReportService>>();

        // Factory retorna novo generator cada vez
        mockFactory
            .Setup(f => f.Create(It.IsAny<string>(), null))
            .Returns(new Mock<IReportGenerator>().Object);

        var service = new ReportService(
            mockFactory.Object,
            mockMetrics.Object,
```



```

    mockLogger.Object
);

// Act
service.GenerateReport("pdf", new ReportData());
service.GenerateReport("pdf", new ReportData());
service.GenerateReport("pdf", new ReportData());

// Assert
// Factory foi chamado 3 vezes - novo generator cada vez
mockFactory.Verify(f => f.Create("pdf", null), Times.Exactly(3));
mockMetrics.Verify(m => m.RecordReportGeneration("pdf"), Times.Exactly(3));
}
}

```

Estes testes são limpos, determinísticos e isolados. Não há dependência de ordem de execução. Não há estado compartilhado entre testes. Cada teste pode ser executado independentemente e produzir o mesmo resultado toda vez.

Benefícios Alcançados

A refatoração para factory stateless traz benefícios substanciais em corretude, manutenibilidade e simplicidade.

Primeiro, thread-safety foi alcançada. O factory não tem estado mutável, então não há possibilidade de race conditions. Múltiplas threads podem chamar Create simultaneamente sem nenhuma coordenação ou sincronização necessária. O código é correto por construção, não porque implementamos sincronização complexa, mas porque eliminamos a necessidade de sincronização removendo estado compartilhado.

Segundo, testabilidade melhorou drasticamente. Testes são agora determinísticos e independentes. Você pode executar testes em qualquer ordem, em paralelo, quantas vezes quiser, e sempre obter os mesmos resultados. Não há estado sendo compartilhado entre testes. Cada teste começa com um slate limpo.

Terceiro, responsabilidades foram claramente separadas. O factory tem uma responsabilidade: criar generators. O componente de métricas tem uma responsabilidade: tracking de estatísticas. Cada componente pode ser desenvolvido, testado e mantido independentemente. Mudanças em como métricas são rastreadas não afetam o factory, e vice-versa.

Quarto, o código se tornou mais simples e mais fácil de entender. A factory stateless é trivialmente simples - apenas um switch que mapeia strings para tipos. Não há lógica complexa de gerenciamento de estado. Não há worries sobre thread-safety dentro do factory em si. A simplicidade aumenta a confiabilidade.

Quinto, performance pode até melhorar. Sem a necessidade de sincronizar acesso a estado compartilhado, não há contenção de locks. Múltiplas threads podem chamar o factory simultaneamente sem se bloquearem mutuamente. Em sistemas de alta concorrência, isto pode resultar em throughput significativamente melhor.

Quando Estado em Factory Pode Ser Aceitável

Embora tenhamos enfatizado fortemente que factories devem ser stateless, honestidade requer que reconheçamos exceções muito raras onde algum estado pode ser aceitável.

Se você está implementando um pool de objetos caros de criar (como conexões de banco de dados, sockets de rede, etc.), o factory pode manter um pool de objetos reutilizáveis. Mas neste caso, você deve usar bibliotecas de pooling estabelecidas e thread-safe, não implementar seu próprio pool do zero. E tecnicamente, este não é realmente um factory no sentido clássico - é mais um pool manager que por acaso usa interface similar a factory.

Se você está implementando caching genuinamente necessário por razões de performance, use estruturas thread-safe como `ConcurrentDictionary` e `Lazy<T>` como demonstrado. Mas primeiro, questione se você realmente precisa de cache. Measure before you optimize. Muitas vezes, criar novos objetos é rápido o suficiente.

Se você está registrando eventos para fins de debugging ou diagnóstico (não para lógica de negócio), logging mínimo pode ser aceitável. Mas use um logger thread-safe fornecido pelo framework, não role sua própria solução de logging.

A regra geral permanece: factories devem ser stateless. Se você acha que precisa de estado, questione profundamente essa necessidade. Na maioria dos casos, a responsabilidade que você está tentando colocar no factory pertence melhor em um componente separado dedicado. Separação de responsabilidades não é apenas teoria acadêmica - ela leva a código mais correto, mais testável e mais mantível.

[Continuarei com os Cenários 4 e 5, a atividade prática Code Smell Detective, e material de conclusão no próximo arquivo devido ao limite de comprimento...]